



## PROGRAMMATION FONCTIONNELLE via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique  
UPR des sciences  
Université de Caen Normandie



### Récursivité



- Les langages de programmation **purement fonctionnels n'ont pas d'instruction de répétition**

⇒ Pas de **for** ni de **while**

- La **seule manière** d'implémenter une répétition est la **récursivité**

⇒ ou principe de récurrence en math

- De manière informelle, une définition (*p.ex.*  $D$ ) est **récursive** quand elle utilise le **nom** ( $D$ ) qu'elle est en train de définir

Définition "**descendant**" d'un individu : est l'un de ses enfants ou un **descendant** de l'un de ses enfants.

A. Ouali

Programmation fonctionnelle en Haskell

2 / 32

### Récursivité : exemple formel



- La définition de la fonction somme :

$$\text{somme} : \mathbb{N} \rightarrow \mathbb{N}$$

$$x \mapsto \begin{cases} 0, & \text{Si } x = 0 \\ x + \text{somme}(x-1), & \text{Si } x > 0 \end{cases}$$

### Récursivité : exemple formel



- La définition de la fonction somme :

$$\text{somme} : \mathbb{N} \rightarrow \mathbb{N}$$

$$x \mapsto \begin{cases} 0, & \text{Si } x = 0 \\ x + \text{somme}(x-1), & \text{Si } x > 0 \end{cases}$$

- **Forme générale**

$$f : D \rightarrow C$$

$$x \mapsto \begin{cases} e_0, & \text{Si } x \in D_0 \\ F(f(e_1), f(e_2), \dots, f(e_k)), & \text{Si } x \in D' \end{cases}$$

-  $e_i$  est une expression qui dépend uniquement de  $x$

-  $F(f(e_1), f(e_2), \dots, f(e_k))$  dépend uniquement de  $x$  et des  $f(e_i)$

-  $D_0 \cup D' = D$  et  $D_0 \cap D' = \emptyset$

### Récursivité : exemple formel



- La définition de la fonction somme :

$$\text{somme} : \mathbb{N} \rightarrow \mathbb{N}$$

$$x \mapsto \begin{cases} 0, & \text{Si } x = 0 \\ x + \text{somme}(x-1), & \text{Si } x > 0 \end{cases}$$

- **Forme générale**

$$f : D \rightarrow C$$

$$x \mapsto \begin{cases} e_0, & \text{Si } x \in D_0 \\ F(f(e_1), f(e_2), \dots, f(e_k)), & \text{Si } x \in D' \end{cases}$$

▸  $f = \text{somme}$

▸  $D = C = \mathbb{N}$

▸  $e_0 = 0$  et  $D_0 = 0$

▸  $k = 1, e_1 = x - 1, F(\text{somme}(e_1)) = \text{somme}(e_1) + x$  et  $D' = \mathbb{N}^*$

### Récursivité : exemple formel



- Fonction somme récursive utilisant les gardes :

```
1 somme x
2
3 | x < 0 = error "La fonction somme n'est définie sur des valeurs négatives."
4
5 | x == 0 = 0 -- case de base : initialisation de la récursivité
6
7 | x > 0 = x + somme (x - 1) -- cas récursif (hérité : calcul à partir de paramètres
8                               -- plus "petits" effet domino)
```

### Récursivité : Base et Hérité



- Deux types d'algorithmes récursifs :

- algorithmes récursifs qui se terminent

```
1 ftermine n
2 | n <= 2 = 0
3 | otherwise = 0.5 * ftermine(n-1) + 2
```

### Récursivité : Base et Hérité



- Deux types d'algorithmes récursifs :

- algorithmes récursifs qui se terminent

```
1 fterminepas n
2 | n <= 2 = 0
3 | otherwise = 0.5 * fterminepas(n-1) + 2
```

- algorithmes récursifs qui ne se terminent pas

```
1 fterminepas n
2 | n == 2 = 0
3 | otherwise = 0.5 * fterminepas(n-3) + 2
```

A. Ouali

Programmation fonctionnelle en Haskell

5 / 32

A. Ouali

Programmation fonctionnelle en Haskell

5 / 32

Définition d'une fonction récursive



- **Décomposition par cas** pour identifier :
  - ⇒ des **sous-problèmes** de même nature
  - ⇒ de **taille inférieure**.
- **Résoudre** le problème entier en :
  - ⇒ supposant que ces **sous-problèmes** résolus par autant d'**appels récurifs**
  - ⇒ combinant les solutions partielles

Définition d'une fonction récursive



- **Décomposition par cas** pour identifier :
  - ⇒ des **sous-problèmes** de même nature
  - ⇒ de **taille inférieure**.
- **Résoudre** le problème entier en :
  - ⇒ supposant que ces **sous-problèmes** résolus par autant d'**appels récurifs**
  - ⇒ combinant les solutions partielles
- Reprenant la fonction somme :
  - $x$  ne peut avoir des valeurs négatives, sinon erreur !
  - si  $x = 0$ , alors la solution est 0.
  - si  $x > 0$ , alors le calcul de  $0 + 1 + \dots + (x - 1)$  est un sous-problème de même nature :
    - ⇒ il suffit de résoudre  $somme(x - 1)$
  - Si l'on suppose que  $somme(x - 1)$  est résolu, alors il suffit de faire  $somme(x - 1) + x$  pour résoudre  $somme(x)$

Correction



- Écrire une **preuve par induction** pour démontrer programme récursif est correct
  - Structure de la preuve par induction
    - Soit  $P(n)$  une propriété d'un entier naturel  $n$ 
      - **Cas de base** :  $P(0)$  est vrai
      - **Cas récursif** : Pour tout  $n \geq 0$ ,  $P(n)$  implique  $P(n + 1)$
      - Si les deux cas sont vérifiés alors  $P(n)$  est vraie pour tout  $n \in \mathbb{N}$  :
  - Reprenant la fonction somme :
- ```
1  somme x
2  | x <= 0 = 0 -- case de base
3  | x > 0 = somme (x - 1) + x -- cas récursif (Hérédité)
```
- La fonction somme est correcte ssi  $somme(n) = 0 + 1 + \dots + (n - 1) + n$  est vrai pour tout  $n \in \mathbb{N}$

Correction



- **Cas de base** :  $somme(0) = 0$  (la ligne 2 sera évaluée)
  - **Cas récursif** : Soit  $n \geq 0$ . si  $somme(n) = 0 + 1 + \dots + n$ , alors  $somme(n + 1) = 0 + 1 + \dots + n + (n + 1)$
- la ligne 3 sera évaluée :  
 $somme(n+1) = (somme\ n) + (n+1)$   
 $somme(n+1) = 0 + 1 + \dots + n + (n+1)$
- Donc,  $somme(n) = 0 + 1 + \dots + n$  pour tout  $n \in \mathbb{N}$
  - On construit le programme en même temps que ça preuve

Définition d'une fonction récursive : exemple



- Définir la fonction `sommeChiffres` qui calcule la somme des chiffres décimaux d'un entier naturel  $n$ .
- Par exemple :

```
1  Prelude> sommeChiffres 123
2  6
```

Récurivité : déroulement



Calcul de factorielle

- Deux définitions équivalentes :

Récurivité : déroulement



Calcul de factorielle

- Deux définitions équivalentes :

```
1  ►
2  fact, fact2 :: Integer -> Integer
3
4  fact n = if (n == 0) then 1
5           else n * (fact (n - 1))
6
7  -- en utilisant les gardes
8
9  fact2 n
10 | n == 0 = 1
11 | otherwise = n * fact2 (n - 1)
12
```

Récurivité : déroulement



```
1  fact :: Integer -> Integer
2  fact n = if (n == 0) then 1
3           else n * (fact (n - 1))
```

Appliquer `fact 4` :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 1 :

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 2 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
===on a besoin de fact 2 :  
====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond  
=====fact 0 vaut 1

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
===on a besoin de fact 2 :  
====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond  
=====fact 0 vaut 1  
=====on multiplie par 1, fact 1 vaut 1

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
===on a besoin de fact 2 :  
====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond  
=====fact 0 vaut 1  
=====on multiplie par 1, fact 1 vaut 1  
=====on multiplie par 2, fact 2 vaut 2

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
===on a besoin de fact 2 :  
====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond  
=====fact 0 vaut 1  
=====on multiplie par 1, fact 1 vaut 1  
=====on multiplie par 2, fact 2 vaut 2  
=====on multiplie par 3, fact 3 vaut 6

Récurtivité : déroulement



```
1 fact :: Integer -> Integer
2 fact n = if (n == 0) then 1
3         else n * (fact (n - 1))
```

Appliquer fact 4 :

==la définition "fact n = n \* fact (n-1)" est celle qui correspond  
==on a besoin de fact 3 :  
===la définition "fact n = n \* fact (n-1)" est celle qui correspond  
===on a besoin de fact 2 :  
====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
====on a besoin de fact 1 :  
=====la définition "fact n = n \* fact (n-1)" est celle qui correspond  
=====on a besoin de fact 0 :  
=====la définition "fact 0 = 1" est celle qui correspond  
=====fact 0 vaut 1  
=====on multiplie par 1, fact 1 vaut 1  
=====on multiplie par 2, fact 2 vaut 2  
=====on multiplie par 3, fact 3 vaut 6  
==on multiplie par 4, fact 4 vaut 24

Récurtivité



► Comment tracer les appels et les rappels (callbacks) d'une fonction récursive en Haskell ?

Récursion terminale



► Prenons la fonction suivante qui donne le plus petit diviseur d'un nombre *n* plus grand ou égal à un autre nombre *d*

```
1 d `divise` n = (mod n d) == 0
2
3 petitDiviseurPlusGrandQue n d
4   | d `divise` n = d
5   | otherwise   = petitDiviseurPlusGrandQue n (d+1)
```

► Cette définition récursive possède une propriété intéressante

⇒ Récursive terminale

Récursion terminale



```
1 petitDiviseurPlusGrandQue n d
2   | d `divise` n = d
3   | otherwise   = petitDiviseurPlusGrandQue n (d+1)
```

Appliquer petitDiviseurPlusGrandQue 35 2 :

====2 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 3

## Récursion terminale

Réursion terminale



```
1 petitDiviseurPlusGrandQue n d
2   | d `divise` n = d
3   | otherwise   = petitDiviseurPlusGrandQue n (d+1)
```

Appliquer petitDiviseurPlusGrandQue 35 2 :

====2 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 3  
====appliquer petitDiviseurPlusGrandQue 35 3 :  
====3 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 4  
====appliquer petitDiviseurPlusGrandQue 35 4 :  
====4 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 5  
====appliquer petitDiviseurPlusGrandQue 35 5 :  
====5 divise 35, on retourne 5  
====on retourne le résultat : 5  
====on retourne le résultat : 5

Réursion terminale



```
1 petitDiviseurPlusGrandQue n d
2   | d `divise` n = d
3   | otherwise   = petitDiviseurPlusGrandQue n (d+1)
```

Appliquer petitDiviseurPlusGrandQue 35 2 :

====2 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 3  
====appliquer petitDiviseurPlusGrandQue 35 3 :  
====3 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 4  
====appliquer petitDiviseurPlusGrandQue 35 4 :  
====4 ne divise pas 35, on doit appliquer petitDiviseurPlusGrandQue 35 5  
====appliquer petitDiviseurPlusGrandQue 35 5 :  
====5 divise 35, on retourne 5  
====on retourne le résultat : 5  
====on retourne le résultat : 5

Aucune nécessité pour retenir des informations sur la pile pour savoir ce que la fonction doit faire après l'appel

Réursion terminale



```
1 petitDiviseurPlusGrandQue n d
2   | d `divise` n = d
3   | otherwise   = petitDiviseurPlusGrandQue n (d+1)
```

Le compilateur est capable d'appliquer une optimisation

Appliquer petitDiviseurPlusGrandQue 35 2 :

==2 ne divise pas 35, on applique petitDiviseurPlusGrandQue 35 3  
==3 ne divise pas 35, on applique petitDiviseurPlusGrandQue 35 4  
==4 ne divise pas 35, on applique petitDiviseurPlusGrandQue 35 5  
==5 divise 35, donc le résultat est 5

Définition d'une fonction récursive : exemple



Redéfinir la fonction factorielle avec une récursivité terminale

Récursivité et filtrage par motifs



Factorielle

une 3-ème définition elle aussi équivalente :

```
1 fact, fact3 :: Integer -> Integer
2
3 fact n = if (n == 0) then 1 else n * (fact (n - 1))
4
5 -- en utilisant le pattern matching (filtrage par motifs) sur les entiers
6
7 fact3 0 = 1 -- <---- ordre des définition est important pour le filtrage
8 fact3 n = n * fact3 (n - 1)
9
10
```

Filtrage par motif ou Pattern matching



- [] est le pattern (modèle) de la liste vide
- (x:xs) est le pattern (modèle) des listes non vides

Exemples

- [1,3,2] vs (x:xs) -> x=1, xs=[3,2]  
car [1,3,2]=1:[3,2]
- [3] vs (x:xs) -> x=3, xs=[]  
car [3]=3:[]

Pattern matching



- [] est le pattern (modèle) de la liste vide
- (x:xs) est le pattern (modèle) des listes non vides

Exemples

- [1,3,2] vs (x:y:xs) -> x=1, y=3, xs=[2]  
car [1,3,2]=1:3:[2]
- [1,1,2] vs (x:y:xs) -> x=1, y=1, xs=[2]  
car [1,1,2]=1:1:[2]
- [3] ne satisfait pas le pattern (x:y:xs)

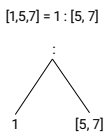
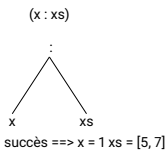
Pattern matching



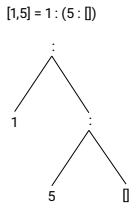
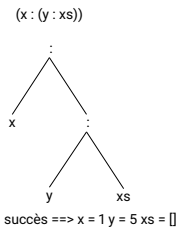
- [] est le pattern de la liste vide
- (x:xs) est le pattern des listes non vides
- (x:xs) est le pattern des listes ayant au moins un élément
- (\_:xs) le premier n'a pas de liaison, le reste des éléments est lié xs
- (x:y:xs) est le pattern des listes ayant au moins deux éléments

- Pour déterminer si une liste possède 2 premiers éléments identiques, peut-on utiliser le pattern (x:x:xs) ?
- Quelles listes représente le pattern [x] ?
- Quelles listes représente le pattern [x, y] ?

Retour sur le filtrage par motif



Retour sur le filtrage par motif



Filtrage par motif : attrape-tout  $\_$



Longueur d'une liste

- polymorphisme grâce aux variables de type
- elle s'applique sur [Int], [Bool], [Char], listes de listes, listes de ce que l'on veut ...

```
> length [3..12] ==> 10
> length [] ==> 0
> length "hello world" ==> 11
> length [[2,5], [1,-3,-12], [7..23]] ==> 3
> length ["un", "deux", "trois"] ==> 3
```

- son type est  $([a] \rightarrow \text{Int})$  où  $a$  est une variable de type <sup>1</sup>.

1. une variable de type permet de désigner n'importe quel type.

Traitement récursif de listes

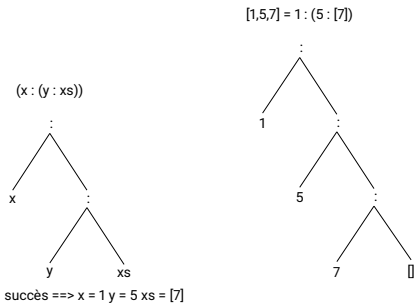


Somme d'une liste d'entiers

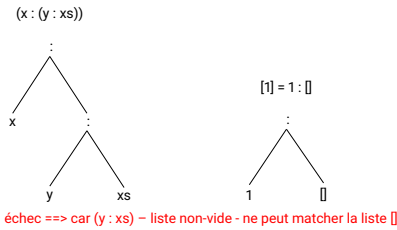
- type de cette fonction :  $[Int] \rightarrow Int$
  - deux définitions équivalentes :
- ```
sum1 :: [Int] -> Int
sum1 l = if (l == [])
  then 0
  else (head l) + (sum1 (tail l))

-- pattern matching
sum3 :: [Int] -> Int
sum3 [] = 0
sum3 (x:xs) = x + (sum3 xs)
```

Retour sur le filtrage par motif



Retour sur le filtrage par motif



Filtrage par motif : attrape-tout  $\_$



- pattern matching en utilisant 2 modèles de listes

```
lg3 [] = 0
lg3 (x:xs) = 1 + (lg3 xs)
```

- la variable  $x$  n'apparaît pas dans la partie droite de la 2nde équation, on peut remplacer  $x$  par l'attrape-tout noté  $\_$

```
lg4 [] = 0
lg4 (_:xs) = 1 + (lg4 xs)
```

Traitement récursif de listes



Longueur d'une liste

- son type :  $[a] \rightarrow Int$
  - deux définitions équivalentes :
- ```
lg1 liste = if (liste == [])
  then 0
  else 1 + lg1 (tail liste)

-- pattern matching
lg3 [] = 0
lg3 (x:xs) = 1 + (lg3 xs)
```



Longueur d'une liste

- son type : [a] -> Int
- pattern matching en utilisant 2 modèles de listes

```
lg3 [] = 0
lg3 (x:xs) = 1 + (lg3 xs)
```
- la variable x n'apparaît pas dans la partie droite de la 2nde équation, on peut remplacer x par l'attrape-tout noté \_

```
lg4 [] = 0
lg4 (_,xs) = 1 + (lg4 xs)
```



Nombres de Fibonacci

- définitions équivalentes :
- ```
1 fibo, fibo2 :: Integer -> Integer
2
3 fibo n = if (n == 0) || (n == 1) then 1
4         else fibo (n-1) + fibo (n-2)
5
6 -- en utilisant les gardes
7
8 fibo2 n
9   | n == 0   = 1
10  | n == 1   = 1
11  | otherwise = fibo2 (n-1) + fibo2 (n-2)
```



Concaténation de deux listes : opérateur infix (++)

- exemples

```
append [] [1,2] = [] ++ [1,2] ==> [1,2]
append [1,2,3] [4,5,6] = [1,2,3] ++ [4,5,6] ==> [1,2,3,4,5,6]
```
- définition

```
append :: [a] -> [a] -> [a]

append xs [] = xs
append [] ys = ys
append (x:xs) ys = x : (append xs ys)
```



Nombres de Fibonacci

- une troisième définition équivalente :
- ```
1 fibo2, fibo3 :: Integer -> Integer
2
3 fibo2 n
4   | n == 0   = 1
5   | n == 1   = 1
6   | otherwise = fibo2 (n-1) + fibo2 (n-2)
7
8 -- en utilisant le pattern matching sur les entiers
9
10 fibo3 0 = 1
11 fibo3 1 = 1
12 fibo3 n = fibo3 (n-1) + fibo3 (n-2)
```